BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 02

> **Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

> To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: `IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward`

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
   Example Logic: `IF wall_ahead AND left_is_visited THEN turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

> Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

It is practically impossible due to a problem known as combinatorial explosion. A mathematically perfect Table-Driven Agent requires a pre-calculated lookup table containing an entry for every single possible sequence of percepts it could ever encounter. For a complex environment like Chess, the number of possible game states and percept histories is astronomically large (exceeding 10^{40}). As the agent's lifetime increases, the size of this table grows exponentially with time. There is neither enough physical memory in the universe to store a table of this magnitude nor enough computing power to search through it. Therefore, AI relies on algorithmic "Agent Programs" rather than exhaustive lookup tables.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

The specific lines representing Condition-Action Rules are located inside the sense_and_act method:

```
if percept["wall_ahead"]:      # The Condition (If)
    return 'Turn_Left'         # The Action   (Then)
```

In Lecture 02, a Condition-Action rule is defined as a direct mapping from a current sensory input to an actuator output. In this code, the if statement acts as the Condition, checking the immediate sensory data (percept["wall_ahead"]). If this condition evaluates to True, the agent immediately executes the return 'Turn_Left' statement, which acts as the Action. It does this directly without consulting any past memory or planning for the future.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The infinite loop is the direct result of combining a blind environment with an amnesiac agent.

Because of Partial Observability, the agent cannot see the overall U-shape of the trap; its sensors only allow it to perceive the single cell directly in front of it.

Because it lacks a Percept History (No internal memory), the agent's brain "resets" every single step. When it hits the first wall, it turns left and moves forward. When it hits the next wall, it perceives the exact same local state (wall_ahead = True). Since it cannot remember that it was just in this situation a few steps ago, it naively applies the exact same Condition-Action rule (Turn_Left), trapping it in an endless, repeating cycle.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent`. Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

The Sensor Model: This is handled by how the agent reads and interprets the percept dictionary (e.g., percept.get('wall_ahead')). The agent uses this to understand its immediate local state (whether the path is currently blocked or clear).

The Transition Model: This is handled by the agent's internal variables: self.my_pos, self.facing, and self.visited_cells. When the agent executes an action like Move_Forward or Turn_Left, it mathematically calculates how that action changes its position and orientation in the unobserved world. By appending these new coordinates to self.visited_cells, the agent builds an internal map, modeling how its past actions have evolved its current state. This allows it to deduce when it is trapped (e.g., checking if the left cell is already in visited_cells) and choose a different action to escape.

Finalize and Lock Lab 02 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.

FACULTY OF COMPUTING